



清华大学

Tsinghua University

前排签到

下载并上传 `ibex_work_upload.zip` 到服务器





清华大学

Tsinghua University

From RTL to GDS 设计流程讲解 (下)

华园，程子艺，陈虹

THU-SIC

2023年5月



目录



清华大学
Tsinghua University

- 综合
- 形式验证
- 布局和布线



布局和布线（五）



• 回顾

- PR data init: 输出PR需要的必要文件
- Floor plan: 完成芯片面积、形状的确立, 并明确IO口的摆放
- Power plan: 对芯片加入电源网络(VDD/VSS)规划, 并进行连接性和电源引脚短路检查
- Placement: 将逻辑电路元件(如门、寄存器等)放置在芯片的物理位置上, 并进行电路密度和布线溢出检查、对关键路径进行分析, 为下一步CTS做准备



布局和布线 (五)



- CTS:

- 运行命令

运行此步骤前, 需要确保placement步骤成功执行, 并输出了placement.enc

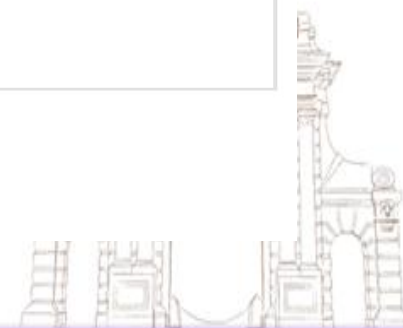
```
make cts
```

- 输入&输出

输入	输出
placement.enc	cts.enc cts.enc.dat cts.cmd cts.log cts.logv cts_timing

- 步骤目标

完成时钟树的综合



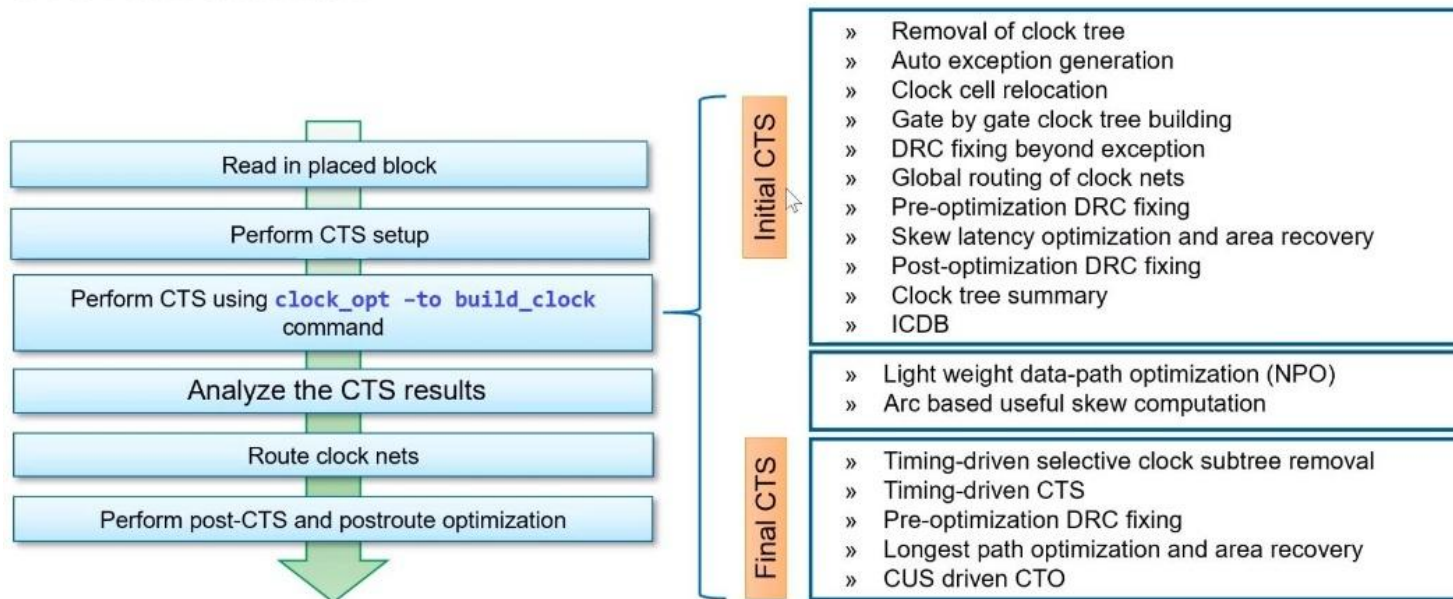
布局和布线 (五)



- CTS:

CTS (Clock Tree Synthesis) 其实就是做一件事情，**从时钟root点开始长Buffer/Inverter tree直至sink点**。而root点是通过create_clock或create_generated_clock来告诉工具的，sink点一部分是设计本身决定的，另外一部分是user defined，它是通过约束文件告诉工具的。

CTS Flow overview



布局 and 布线 (五)



- CTS:

运行脚本讲解 1

```
source $::env(RESULT_DIR)/pr/data/placement.enc 读文件
```

```
# -----
```

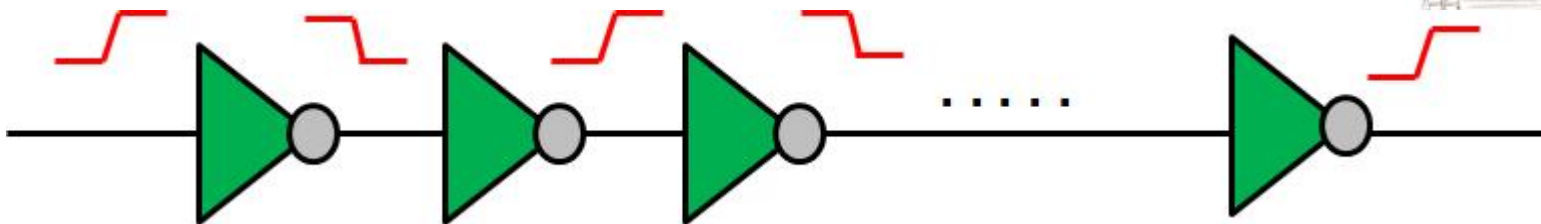
```
# ccopt property setting ccopt是一个时钟树合成工具
```

```
# -----
```

```
set_ccopt_property use_inverters true 使用反相器链来  
优化时钟树
```

```
./scripts/pr/cts.tcl
```

使用 inverter clock tree 的好处是，高低脉冲的宽度是对称的。对于时钟信号来说，这十分关键。特别是对于SOC的时钟信号来说，因为SOC中存在大量的正沿驱动和负沿驱动的flipflops 的交互。

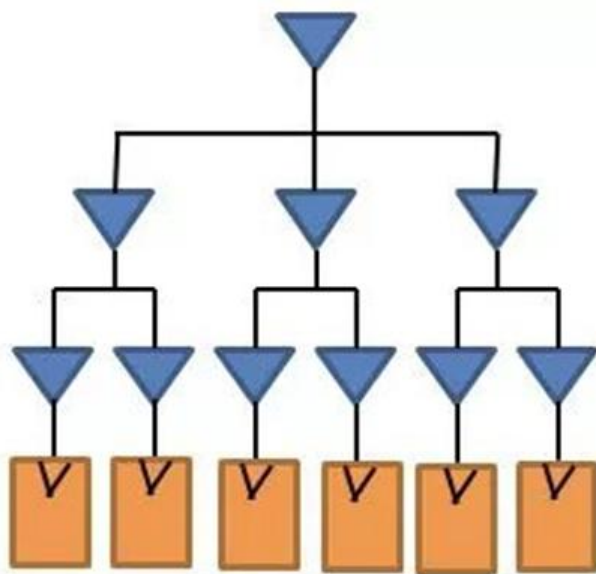


布局 and 布线 (五)

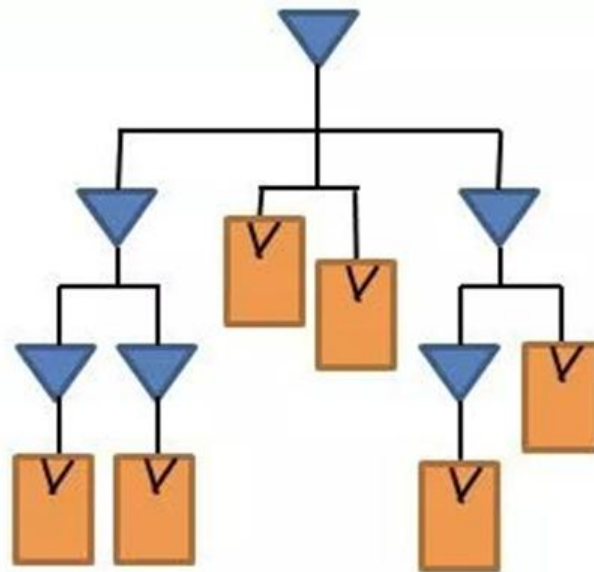


- CTS:

CCD技术: Concurrent Clock Datapath, 并行优化 clock 和 data path, 基于innovus引擎CCOpt (Clock Concurrent Optimization)



传统CTS



CCD

CCOpt 将CTS和postCTS的timing optimize集合为一体, 在优化data path的同时进行clock path的优化。根据timing情况自动调整时钟树各点的延迟, 以达到合理利用skew来优化timing的目的

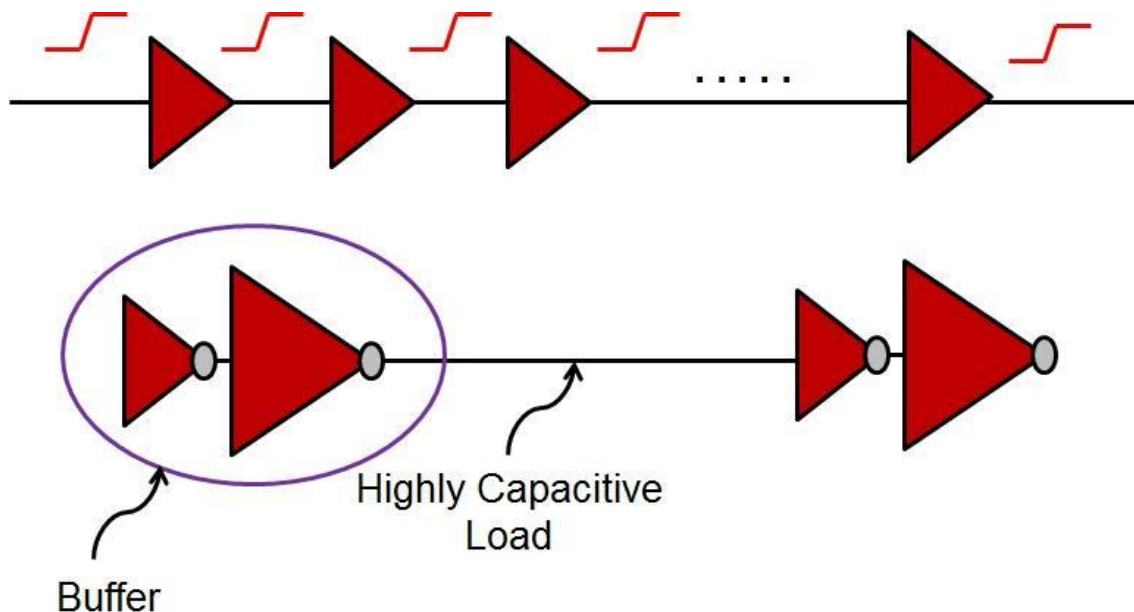
布局 and 布线 (五)



- CTS:

除此之外，还有一种基于**buffer**的时钟树

理论上，buffer是由两个完全相同的inverter级联而成，但这不是标准库单元中设计buffer的做法。为了节省面积，buffer的第一级通常驱动很小，并且离第二级 inverter 很近，而第二级 inverter 的驱动力更大。



布局和布线 (五)



- CTS:

运行脚本讲解 2

```
# ----- 用来设置时钟树合成过程中使用的反相器库 -----  
# set cts opt use cell  
# -----  
set cts_inv_cells [split $::env(CTS_INV_CELL)]  
foreach libraryname $cts_inv_cells {  
    puts $libraryname  
    echo "setDontUse $libraryname false"  
    setDontUse $libraryname false  
}  
set_ccopt_property inverter_cells [get_lib_cells "$::env(CTS_INV_CELL)"]
```

将环境变量 “CTS_INV_CELL” 中的值分割成一个列表，并将其赋值给变量 “cts_inv_cells”

反相器库列表中的每个库执行以下操作

打印库名称

打印设置 “setDontUse” 命令的字符串

设置库 “libraryname” 可用

设置使用的反相器库

./scripts/pr/cts.tcl

这段代码的作用是将指定的反相器库设置为时钟树合成过程中可用的库。其中，“setDontUse” 命令用于设置库的可用性，它的第二个参数是一个布尔值，用于指定库是否可用。在这里，将所有的反相器库都设置为可用，以便在时钟树合成过程中使用。最后，将使用的反相器库设置为时钟树合成工具ccopt使用的反相器库。

布局和布线 (五)



- CTS:

运行脚本讲解 3

```
# -----  
# clk net routing ndr setting 用于设置时钟网络的非默认规则 (NDR) 和时钟网络的路由规则  
# -----  
  
set mul $::env(CTS_ROUTING_MUL) 将环境变量 "CTS_ROUTING_MUL" 的值赋值给变量 "mul"  
set ndr_min_layer $::env(NDR_CTS_MIN_LAYER) 将环境变量 "NDR_CTS_MIN_LAYER" 的值赋值给变量 "ndr_min_layer"  
set ndr_max_layer $::env(NDR_CTS_MAX_LAYER) 将环境变量 "NDR_CTS_MAX_LAYER" 的值赋值给变量 "ndr_max_layer"  
set routing_top_layer [lindex [split $::env(CTS_ROUTING_LAYER_RANGE)] 0]  
set routing_bottom_layer [lindex [split $::env(CTS_ROUTING_LAYER_RANGE)] 1]  
将环境变量 "CTS_ROUTING_LAYER_RANGE" 分割成一个列表, 并将其第一个元素赋值给变量 "routing_top_layer" 将其第二个元素赋值给变量 "routing_bottom_layer"
```

./scripts/pr/cts.tcl

NDR的全称是Non-Default Rule, 即非默认规则。在时钟树合成中, NDR用于指定时钟信号的布局和路由规则, 以满足时序约束和电路性能要求。NDR通常由宽度乘数、间距乘数和直通层等参数组成, 用于控制时钟网络的布局和路由。时钟树合成工具通常会自动计算和生成NDR, 但也可以手动设置和调整以优化时钟树设计。

布局和布线 (五)



• CTS: 运行脚本讲解 4

`add_ndr -name cts_1 \` 添加一个名为“cts_1”的NDR，并设置其宽度和间距的乘数

`-width_multiplier "$ndr_min_layer:$ndr_max_layer $mul" \`

设置NDR的宽度乘数参数，其中“\$ndr_min_layer:\$ndr_max_layer”表示NDR的作用层范围，即从层“ndr_min_layer”到层“ndr_max_layer”，“\$mul”表示宽度乘数的值

`-spacing_multiplier "$ndr_min_layer:$ndr_max_layer $mul“`

设置NDR的间距乘数参数，其中“\$ndr_min_layer:\$ndr_max_layer”表示NDR的作用层范围，即从层“ndr_min_layer”到层“ndr_max_layer”，“\$mul”表示间距乘数的值。

`create_route_type -name clk_net_rule \`

`-non_default_rule cts_1 \`

`-top_preferred_layer $ndr_min_layer \`

`-bottom_preferred_layer $ndr_max_layer`

创建一个名为“clk_net_rule”的路由规则，并将NDR“cts_1”设置为非默认规则。还设置了首选层和底部首选层。

`set_ccopt_property route_type clk_net_rule -net_type trunk`

`setNanoRouteMode -quiet \`

`-routeTopRoutingLayer $routing_top_layer \`

`-routeBottomRouting $routing_bottom_layer`

路由规则“clk_net_rule”的网络类型设置“trunk”表示该规则适用于时钟干线(clock trunk)

设置NanoRoute的模式，并指定顶部路由层和底部路由层，以控制时钟网络的布局和路由。其中，“quiet”选项关闭了输出信息，以减少输出内容。顶部路由层和底部路由层的设置可以根据电路的性能要求和时序约束进行调整。

`./scripts/pr/cts.tcl`

布局和布线（五）



- **Clock Trunk:**

时钟干线 (Clock Trunk) 是指用于传输时钟信号的主要信号路径，通常是由多个时钟分配器 (Clock Tree Synthesis, CTS) 输出的时钟信号组成的网络。时钟干线的设计和布局对于整个电路的性能和时序约束非常重要，因为时钟信号的稳定性和相位关系直接影响电路的工作速度和功耗。因此，在电路设计中，需要对时钟干线进行特殊的处理和优化，包括使用NDR (Non-default Routing) 进行时钟布线、设置时钟规则进行路由等。



布局和布线 (五)



- **NanoRoute:**

NanoRoute是一种自动布线工具，可以根据电路的特定要求和约束进行自适应布线，以提高电路的性能和可靠性。NanoRoute的模式包括以下几种：

`setNanoRouteMode -normal`: 正常模式，用于普通布线。

`setNanoRouteMode -fast`: 快速模式，用于快速布线，但可能会牺牲一些性能。

`setNanoRouteMode -highDensity`: 高密度模式，用于高密度电路的布线。

`setNanoRouteMode -congestionDriven`: 拥塞驱动模式，用于处理电路中的拥塞现象。

`setNanoRouteMode -globalRouting`: 全局路由模式，用于处理大规模电路的布线。

`setNanoRouteMode -quiet` 关闭输出信息，以减少输出内容，使得布线的过程更加简洁和高效。



布局和布线（五）



- CTS： 运行脚本讲解 5

```
# -----  
# create the cts 创建时钟树 (Clock Tree) 的规范，并读取规范文件中的内容  
# -----  
create_ccopt_clock_tree_spec -file $::env(RESULT_DIR)/pr/data/clk.spec  
source $::env(RESULT_DIR)/pr/data/clk.spec
```

创建时钟树的规范文件，并将其保存在指定的文件路径中
读取时钟树规范文件中的内容，并将其加载到当前的环境中。

./scripts/pr/cts.tcl

这段代码的作用是创建时钟树的规范，并将其保存到文件中。规范文件中包含了时钟树的各种参数和约束，如时钟频率、时钟延迟、时钟偏移等，以便后续的布线和优化。读取规范文件的目的是为了后续的时钟树合成和布线过程中使用。



布局和布线 (五)



- CTS: 运行脚本讲解 6

```
# -----  
# run ccopt 运行CCOpt工具对电路进行时钟优化和布线, 并生成相关的时序报告和统计信息  
# -----  
ccopt_design -cts 运行CCOpt  
report_ccopt_skew_groups 生成时钟偏移组的统计信息和报告  
timeDesign -postCTS \  
-pathReports \  
-drvReports \  
-slackReports \  
-numPaths 50 \  
-prefix postCTS \  
-outDir $::env(RESULT_DIR)/pr/report/cts_timing 保存  
  
saveDesign $::env(RESULT_DIR)/pr/data/cts.enc  
#exit
```

./scripts/pr/cts.tcl

布局和布线（六）



- `post_cts_opt`:

```
make post_cts_opt
```

对经过时钟优化和布线后的电路进行进一步的优化，
并生成相关的时序报告和统计信息

Post-cts更加精确：

在pre-CTS的时候，我们将时钟的不确定性设定为target的skew和jitter值之和来模拟真实的时钟；而post-CTS之后，时钟树propagate delay已经确定，skew真实存在，所以uncertainty就是时钟的真实抖动值。



布局和布线 (六)



- Post-CTS: 运行脚本讲解

```
source $::env(RESULT_DIR)/pr/data/cts.enc
```

设置约束模式为交互模式

```
set_interactive_constraint_modes [all_constraint_modes -active]
```

```
set_propagated_clock [all_clocks]
```

设置时钟传播路径

```
setOptMode -fixDrc true -fixFanoutLoad true
```

设置优化模式, 包括解决DRC (Design Rule Check) 错误和Fanout Load问题

```
# -----
```

```
# opt design after CTS
```

```
# -----
```

```
optDesign -postCTS
```

进行时序优化: 时钟优化和布线优化

```
optDesign -postCTS -hold
```

```
saveDesign $::env(RESULT_DIR)/pr/data/post_cts_opt.enc
```

输出

```
timeDesign -postCTS -pathReports -drvReports -slackReports -numPaths 50 -prefix
```

```
postCTS -outDir $::env(RESULT_DIR)/pr/report/cts_opt_timing
```

生成报告

```
#exit
```

./scripts/pr/post_cts_opt.tcl

布局和布线 (六)



- Post-CTS: 报告讲解

```
#####
# Generated by:      Cadence Innovus 19.15-s075_1
# OS:                Linux x86_64(Host ID edaserver)
# Generated on:      Sun May 7 23:54:32 2023
# Design:            ibex_core
# Command:           timeDesign -postCTS -pathReports -drvReports -slackReports -numPaths 50 -prefix postCTS -outDir ./result/pr/report/cts_opt_timing
#####

-----
timeDesign Summary
-----

+-----+-----+-----+-----+-----+-----+-----+
| Setup mode | all | reg2reg | in2reg | reg2out | default | feedthr |
+-----+-----+-----+-----+-----+-----+-----+
| WNS (ns): | 0.003 | 0.003 | 2.010 | 0.768 | 8.283 | 4.691 |
| TNS (ns): | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| Violating Paths: | 0 | 0 | 0 | 0 | 0 | 0 |
| All Paths: | 7863 | 4137 | 5649 | 101 | 1 | 32 |
+-----+-----+-----+-----+-----+-----+-----+

+-----+-----+-----+
| DRVs | Real | Total |
+-----+-----+-----+
| | Nr nets(terms) | Worst Vio | Nr nets(terms) |
+-----+-----+-----+
| max_cap | 0 (0) | 0.000 | 0 (0) |
| max_tran | 0 (0) | 0.000 | 0 (0) |
| max_fanout | 0 (0) | 0 | 0 (0) |
| max_length | 0 (0) | 0 | 0 (0) |
+-----+-----+-----+

Density: 41.024%
Routing Overflow: 1.87% H and 4.47% V
-----
postCTS.summary.gz (END)
```

Placement的报告是:

Density: 40.411%
Routing Overflow: 1.32% H and 3.36% V

原因是加入了时钟树

布局和布线 (六)



- Post-CTS:

```
#####  
# Generated by:      Cadence Innovus 19.15-s075_1  
# OS:               Linux x86_64(Host ID edaserver)  
# Generated on:      Sun May 7 23:54:33 2023  
# Design:           ibex_core  
# Command:          timeDesign -postCTS -pathReports -drvReports -slackReports -numPaths 50 -prefix postCTS -outDir ./resu  
#####  
Path 1: MET Setup Check with Pin if_stage_i/gen_prefetch_buffer.prefetch_buffer_  
i/fifo_i/instr_addr_q_reg[18]/CLK  
Endpoint:  if_stage_i/gen_prefetch_buffer.prefetch_buffer_i/fifo_i/instr_addr_  
q_reg[18]/DE (^) checked with leading edge of 'core_clock'  
Beginpoint: if_stage_i/instr_rdata_id_o_reg[18]/Q  
(^) triggered by leading edge of 'core_clock'  
Path Groups: {reg2reg}  
Analysis View: max view  
Other End Arrival Time          0.007  
- Setup                        0.458  
+ Phase Shift                   17.400  
+ CPPR Adjustment               0.024  
= Required Time                 16.972  
- Arrival Time                 16.969  
= Slack Time                    0.003  
    Clock Rise Edge            0.000  
    + Source Insertion Delay    -0.750  
    = Beginpoint Arrival Time    -0.750
```

可以看到关键路径的时序裕量，
以及每条路径的时序信息



布局和布线（七）



- Routing:

- 运行命令

运行此步骤前，请确保post_cts_opt成功执行，并输出了post_cts_opt.enc

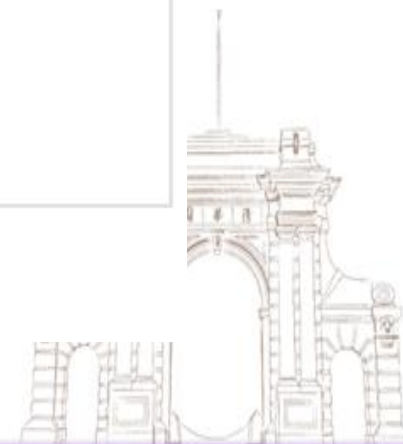
```
make routing
```

- 输入&输出

输入	输出
post_cts_opt.enc	routing.enc routing.enc.dat routing.cmd routing.log routing.logv routing_timing routing_opt_timing

- 步骤目标

完成block的routing，时序满足要求，完成电源地的检查和drc的检查



布局布线（七）



- Routing:

运行脚本讲解 1 这个脚本文件的作用是在芯片设计中的布局布线流程中执行路由设计

```
source $::env(RESULT_DIR)/pr/data/post_cts_opt.enc
```

```
# -----
```

```
# NanoRoute Mode setting 利用NanoRoute工具进行布线
```

```
# -----
```

```
setNanoRouteMode -quiet -routeWithTimingDriven true 使用时序驱动的路由
```

```
setAnalysisMode -analysisType onChipVariation 启用芯片内变异分析
```

```
setNanoRouteMode -quiet -drouteEndIteration 70 设置路由算法的最大迭代次数为70次
```

```
setNanoRouteMode -quiet -drouteFixAntenna true 避免天线效应
```

```
setNanoRouteMode -quiet -drouteUseMultiCutViaEffort medium 设置中等多切割孔优化
```

```
setNanoRouteMode -quiet -drouteMinSlackForWireOptimization 0.1 最小时序裕量为0.1
```

```
setNanoRouteMode -quiet -routeTopRoutingLayer $::env(MAX_ROUTING_LAYER)
```

```
setNanoRouteMode -quiet -routeBottomRoutingLayer $::env(MIN_ROUTING_LAYER)
```

```
setDelayCalMode -engine default -siAware true 设置路由的顶层和底层层数值
```

```
设置延迟计算的引擎为默认值，并启用芯片内变异分析
```

```
./scripts/pr/routing.tcl
```


布局和布线（七）



- 芯片内变异分析：

芯片内变异分析是一种在芯片设计中用于分析不同晶体管之间性能差异的技术。由于制造工艺的不确定性，同一芯片上的不同晶体管可能存在微小的性能差异，这些差异可能会导致芯片的性能和可靠性问题。芯片内变异分析可以帮助设计人员识别这些差异，并采取相应的措施来解决问题。

芯片内变异分析通常是在芯片设计的后期进行的，可以使用各种工具和技术来实现。其中一种常用的方法是使用仿真工具来模拟芯片内的变异情况，以便评估芯片的性能和可靠性。另一种方法是使用实际芯片测试数据来进行分析，以便更准确地评估芯片的性能和可靠性。

setDelayCalMode -engine default -siAware true

通过（1）准确的延迟计算和（2）芯片内变异分析来评估芯片的时序性能



布局和布线（七）



`setNanoRouteMode -quiet -drouteUseMultiCutViaEffort medium`

- 多切割孔优化：

多切割孔优化是一种在芯片设计中的布局布线流程中使用多个切割孔来连接不同层的方法，以提高布线的质量和效率。在芯片设计中，由于不同层之间的电气特性不同，因此需要使用不同的切割孔来连接这些层。多切割孔优化可以在不同层之间使用多个切割孔，以减少布线的长度和电阻，从而提高芯片的性能和功耗效率。



布局和布线（七）



- Routing:

运行脚本讲解 2 这个脚本文件的作用是在芯片设计中的布局布线流程中执行路由设计

```
# -----  
# Route Design  
# -----  
routeDesign -globalDetail 执行路由设计，其中-globalDetail表示使用全局详细路由模式  
saveDesign $::env(RESULT_DIR)/pr/data/routing.enc 保存文件  
  
# -----  
# save Design  
# -----  
timeDesign -postRoute -prefix postRoute -outDir  
$::env(RESULT_DIR)/pr/report/routing_timing
```

执行后路由，并将结果保存到指定的目录中。

-postRoute表示使用后路由模式；

-prefix postRoute表示设置结果文件的前缀为postRoute；

-outDir ... /routing_timing表示将结果保存到指定的目录中；

./scripts/pr/routing.tcl

布局 and 布线 (七)



- Routing:

```
#####  
# Generated by:      Cadence Innovus 19.15-s075_1  
# OS:               Linux x86_64(Host ID edaserver)  
# Generated on:      Mon May 8 00:08:20 2023  
# Design:           ibex_core  
# Command:          timeDesign -postRoute -prefix postRoute -outDir ./result/pr/report/routing_timing  
#####
```

timeDesign Summary

	Setup mode	all	reg2reg	in2reg	reg2out	default	feedthr
	WNS (ns):	-4.121	-4.121	1.186	-1.919	9.034	1.275
	TNS (ns):	-3184.7	-3175.6	0.000	-9.159	0.000	0.000
	Violating Paths:	1584	1575	0	9	0	0
	All Paths:	7863	4137	5649	101	1	32

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	42 (42)	-0.052	42 (42)
max_tran	41 (152)	-0.917	41 (152)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 41.024%

Total number of glitch violations: 3

布局和布线（七）



- Routing:

可能发生glitch的路径在以下线网名字中:

```
NetName                                vlPeak    vlView      vhPeak    vhView
#
n5912  0.3051 (RIP) max_view 0.75474 (RIP) max_view
n6704  0.42318 (RIP) max_view 0.73278 (RIP) max_view
n6759  0.3717 (RIP) max_view 0.72954 (RIP) max_view

Total number of glitch violations: 3
```

可以根据网表文件逐一排查；但是可以等到post-routing优化后再进行排查



布局和布线（八）



- **Post-routing:**

执行routing之后的优化步骤

```
make routing_opt
```

Post-routing是布局布线的第二阶段，也称为detailed routing，它主要是对global routing后的电路进行更加精细的布线。在post-routing阶段，布线工具会对电路进行更加详细的分析，包括对电路的时序、功耗、噪声等方面的考虑。最终目标是优化电路的性能和可靠性。



布局和布线 (八)



- Post-routing:

运行脚本讲解

```
source $::env(RESULT_DIR)/pr/data/routing.enc
```

```
optDesign -postRoute -setup
```

执行post-routing的优化设计，包括时序优化和功耗优化等。

-postRoute选项表示进行post-routing优化，-setup选项表示进行优化设计的设置

```
# -----
```

```
# Save Design 保存文件
```

```
# -----
```

```
timeDesign -postRoute -prefix postRouteOpt -outDir
```

```
$::env(RESULT_DIR)/pr/report/routing_opt_timing
```

```
saveDesign $::env(RESULT_DIR)/pr/data/routing_opt.enc 保存文件
```

计算post-routing优化后的电路的时序性能。

-postRoute选项表示进行post-routing时序分析，

-prefix选项设置时序分析结果的前缀，

-outDir选项设置时序分析结果的输出目录。

```
./scripts/pr/routing_opt.tcl
```

布局 and 布线 (八)



- Post-routing:

```
#####
# Generated by:      Cadence Innovus 19.15-s075_1
# OS:                Linux x86_64(Host ID edaserver)
# Generated on:      Mon May  8 00:15:25 2023
# Design:            ibex_core
# Command:           timeDesign -postRoute -prefix postRouteOpt -outDir ./result/pr/report/routing_opt_timing
#####
```

timeDesign Summary

Setup mode	all	reg2reg	in2reg	reg2out	default	feedthr
WNS (ns):	0.180	0.228	1.197	0.180	9.032	2.099
TNS (ns):	0.000	0.000	0.000	0.000	0.000	0.000
Violating Paths:	0	0	0	0	0	0
All Paths:	7863	4137	5649	101	1	32

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	1 (1)	-0.023	1 (1)
max_tran	1 (19)	-0.272	1 (19)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Post-routing 后消除了
一个glitch冲突

Density: 41.839%
Total number of glitch violations: 2

布局和布线 (八)



- Post-routing:

可能发生glitch的路径在以下线网名字中:

```
NetName                vlPeak    vlView                vhPeak    vhView
#
data_wdata_o[12] 0.46008 (RIP) max_view 0.80028 (RIP) max_view
data_wdata_o[7] 0.1854 (RIP) max_view 0.72234 (RIP) max_view
Total number of glitch violations: 2
```

可以根据信号逐级排查；这就需要前端设计人员判断是否需要改设计了！



布局和布线 (八)



- Post-routing:

可以通过命令 `verifyConnectivity -type all` 检查电源地连接

使用命令 `verify_drc` 检查DRC

```
routing_opt.tcl - KWrite
File Edit View Bookmarks Tools Settings Help
+ New Open Save Save As Close Undo Redo
source $::env(RESULT_DIR)/pr/data/routing.enc
# -----
# OptDesign after routing
# -----
optDesign -postRoute -setup

# -----
# Add filler cell
# -----
#set filler_cell [split $::env(FILL_CELLS)]
#setFillerMode -doDRC true -corePrefix Filler -core $filler_cell
#addFiller

# -----
# Save Design
# -----
timeDesign -postRoute -prefix postRouteOpt -outDir $::env(RESULT_DIR)/pr/report/routing_opt_timing
saveDesign $::env(RESULT_DIR)/pr/data/routing_opt.enc

verifyConnectivity -type all
verify_drc
```

布局和布线 (八)



清华大学
Tsinghua University

- **Post-routing:**

检查电源地连接

```
***** Start: VERIFY CONNECTIVITY *****
```

```
Start Time: Tue May 23 15:49:21 2023
```

```
Design Name: ibex_core
```

```
Database Units: 1000
```

```
Design Boundary: (0.0000, 0.0000) (556.1400, 554.2000)
```

```
Error Limit = 1000; Warning Limit = 50
```

```
Check all nets
```

```
Net boot_addr_i[0]: Found a geometry with bounding box (0.00,-0.00) (0.14,0.49) outside the design boundary.
```

```
Violations for such geometries will be reported.
```

```
**WARN: (IMPVFC-97): IO pin test_si of net test_si has not been assigned. Please make sure it is assigned and rerun verifyConnectivity.
```

```
**WARN: (IMPVFC-97): IO pin test_so of net test_so has not been assigned. Please make sure it is assigned and rerun verifyConnectivity.
```

```
**** 15:49:21 **** Processed 5000 nets.
```

```
**** 15:49:22 **** Processed 10000 nets.
```

```
Net VDD: dangling Wire.
```

```
Net VSS: dangling Wire.
```

```
Begin Summary
```

```
463 Problem(s) (IMPVFC-94): The net has dangling wire(s).
```

```
463 total info(s) created.
```

```
End Summary
```

```
End Time: Tue May 23 15:49:22 2023
```

```
Time Elapsed: 0:00:01.0
```



布局和布线 (八)



- **Post-routing:**

检查DRC

```
Verification Complete : 463 Viols. 0 Wrngs.  
(CPU Time: 0:00:00.7 MEM: 0.000M)
```

```
*** Starting Verify DRC (MEM: 1289.7) ***
```

```
VERIFY DRC ..... Starting Verification  
VERIFY DRC ..... Initializing  
VERIFY DRC ..... Deleting Existing Violations  
VERIFY DRC ..... Creating Sub-Areas  
VERIFY DRC ..... Using new threading  
VERIFY DRC ..... Sub-Area: {0.000 0.000 141.120 138.880} 1 of 16  
VERIFY DRC ..... Sub-Area : 1 complete 697 Viols.  
VERIFY DRC ..... Sub-Area: {141.120 0.000 282.240 138.880} 2 of 16  
VERIFY DRC ..... Sub-Area : 2 complete 192 Viols.  
VERIFY DRC ..... Sub-Area: {282.240 0.000 423.360 138.880} 3 of 16  
VERIFY DRC ..... Sub-Area : 3 complete 90 Viols.  
VERIFY DRC ..... Sub-Area: {423.360 0.000 556.140 138.880} 4 of 16  
VERIFY DRC ..... Sub-Area : 4 complete 18 Viols.  
VERIFY DRC ..... Sub-Area: {0.000 138.880 141.120 277.760} 5 of 16
```

```
**WARN: (IMPVFG-1103): VERIFY DRC did not complete: Number of violations exceeds the Error Limit [1000]
```

```
Verification Complete : 1000 Viols.
```

```
*** End Verify DRC (CPU: 0:00:01.4 ELAPSED TIME: 1.00 MEM: 0.0M) ***
```



布局和布线 (八)



- Chip Finish:

- 运行命令

执行该步骤时, 请确保routing_opt步骤成功执行, 并输出了routing_opt.enc

```
make chip_done
```

- 输入&输出

输入	输出
routing_opt.enc gds.map	chip_done.cmd chip_done.log chip_done.logv chip_done.enc.dat chip_done.enc ibex_core.gds ibex_lvs.vg ibex_routing.vg ibex_routing.def create_text_ibex.tcl hcell_list VDD.pp VSS.pp

- 步骤目标

输出设计的.def,.v,.gdsii文件



布局和布线（九）



- Chip Finish :

运行脚本讲解1

```
source $::env(RESULT_DIR)/pr/data/routing_opt.enc
```

```
# -----
```

```
# Remove the unused net&module
```

```
# -----
```

```
remove_assigns -buffering
```

```
deleteDanglingNet
```

```
deleteEmptyModule
```

```
./scripts/pr/chip_done.tcl
```

这部分代码是用于删除未使用的网络和模块。

remove_assigns -buffering用于删除无效 assign 逻辑赋值。

deleteDanglingNet命令用于删除所有未连接到其他网络的网络。

deleteEmptyModule命令用于删除所有没有子模块或未连接到其他网络的模块。

这些命令可以减少芯片的面积和复杂度。



布局和布线（九）



- Chip Finish:

运行脚本讲解2

```
# -----  
# Re-connect the PG net after add physical cell  
# 用于将PG网络连接到芯片的电源和地 -----  
globalNetConnect VDD -type pgpin -pin {VPB} -inst *  
globalNetConnect VDD -type pgpin -pin {VPWR} -inst *  
globalNetConnect VDD -type tiehi -pin {VPWR} -inst *  
globalNetConnect VDD -type tiehi -pin {VPB} -inst *  
globalNetConnect VDD -type net -net VDD  
globalNetConnect VSS -type pgpin -pin {VGND} -inst *  
globalNetConnect VSS -type pgpin -pin {VNB} -inst *  
globalNetConnect VSS -type tielo -pin {VGND} -inst *  
globalNetConnect VSS -type tielo -pin {VNB} -inst *  
globalNetConnect VSS -type net -net VSS  
verifyConnectivity -type all -error 1000 -warning 50
```

tielo是一个电源或地连接器的类型，用于将电源或地引脚连接到芯片的电源和地平面

./scripts/pr/chip_done.tcl

布局 and 布线 (九)



- Chip Finish:

运行脚本讲解3

defOut命令用于将芯片的布线定义文件写入指定的文件中

-floorplan选项表示输出芯片的布局信息,

-netlist选项表示输出芯片的电路网表, **-routing**选项表示

```
# Write out the routing def
# -----
set lefDefOutVersion 5.8
defOut -floorplan -netlist -routing $::env(RESULT_DIR)/pr/data/ibex_routing.def
```

输出芯片的布线定义。

```
# -----
# write out the spf by QRC
# -----
```

这部分代码用于写出布线定义文件和spf文件

```
setExtractRCMode -engine postRoute
reset_parasitics
extractRC
```

./scripts/pr/chip_done.tcl

setExtractRCMode 命令用于设置从布线中提取RC参数的模式

reset_parasitics 命令用于重置电容和电感等参数

extractRC 命令用于从布线中提取RC参数



布局和布线（九）



- Chip Finish:

运行脚本讲解4 这部分代码用于将芯片的网表写入指定的文件中

```
# Write out the netlist
saveNetlist $::env(RESULT_DIR)/pr/data/ibex_routing.vg
saveNetlist -excludeLeafCell -includePowerGround -flattenBus
$::env(RESULT_DIR)/pr/data/ibex_lvs.vg
```

./scripts/pr/chip_done.tcl

saveNetlist命令用于将芯片的网表保存到指定的文件中。

-excludeLeafCell选项表示排除所有叶子单元

-includePowerGround选项表示包括电源和地连接器

-flattenBus选项表示将总线扁平化为单个信号线。

这个脚本包含两个saveNetlist命令，分别用于保存不包含PG端口和网络的网表（ibex_routing.vg），以及包含PG端口和网络的网表的网表（ibex_lvs.vg）



布局和布线 (九)



- Chip Finish:

运行脚本讲解5

```
# -----  
# Create_text_ibex.tcl is created for adding text when run lvs  
# VDD.pp&VSS.pp is created for run ir-drop analysis  
# -----  
set text_layer_number $::env(LAYER_TEXT_NUM) 设置变量值  
set file [open $::env(SCRIPTS_DIR)/pv/create_text_ibex.tcl w+] 设置脚本文件tcl  
set vdd_ploc_file [open $::env(SCRIPTS_DIR)/ir_v/VDD.pp w+] 设置两个.pp文件，用于  
set vss_ploc_file [open $::env(SCRIPTS_DIR)/ir_v/VSS.pp w+] 之后的IR-drop分析  
  
puts $file "set GDS_FILE \[layout create  
$::env(RESULT_DIR)/pr/data/ibex_core.merge.gds -dt_expand\  
puts $file "\$GDS_FILE create layer $text_layer_number"  
创建一个名为ibex_core.merge.gds的GDS文件，并使用-dt_expand选项对其进行扩展  
使用GDS_FILE变量创建一个新的文本层，并将其写入Tcl脚本文件中  
  
./scripts/pr/chip_done.tcl
```

布局和布线 (九)



- Chip Finish: 运行脚本讲解6

```
set net VDD
set special_wire [dbGet [dbGet top.pgNets.name $net -p].sWires.layer.name
$::env(H_STRIPE_METAL) -p2]
```

这段代码的作用是在指定的特殊金属层上添加VDD电源网的位置信息

```
set count 0
```

```
foreach sw $special_wire {
    set llx [dbGet $sw.box_llx]
    set lly [dbGet $sw.box_lly]
    set urx [dbGet $sw.box_urx]
    set ury [dbGet $sw.box_ury]
    set x [expr ($llx + $urx) / 2.0*1000]
    set y [expr ($lly + $ury) / 2.0*1000]
    set x_ploc [expr ($llx + $urx) / 2.0]
    set y_ploc [expr ($lly + $ury) / 2.0]
```

遍历特殊金属层上的每个电源网段，获取其边界框的坐标信息，并计算出其中心点的坐标。同时，还计算了这个电源网段的中心点在物理布局中的坐标

```
puts $file "\$GDS_FILE create text ibex_core $text_layer_number $x $y $net"
```

```
puts $vdd_ploc_file "VDD_$count $x_ploc $y_ploc $::env(H_STRIPE_METAL)"
```

```
set count [expr $count + 1]
```

这个电源网段的中心点在物理布局中的坐标信息写入VDD.pp文件中；count变量加1，以便为下一个电源网段命名

```
}
```

```
close $vdd_ploc_file
```

./scripts/pr/chip_done.tcl

布局和布线（九）



- Chip Finish：运行脚本讲解7

这段代码的作用是输出用于LVS验证和IR-drop分析的单元格列表

```
# -----
```

```
# output the hcell list for lvs & ir-drop analysis
```

```
# -----
```

```
set hcell_file [open $::env(SCRIPTS_DIR)/pv/hcell_list w]
```

```
set hcell_list_ir [open $::env(SCRIPTS_DIR)/ir_v/cell_list w]
```

```
set cells [dbGet [dbGet top.insts.isPhysOnly 0 -p].cell.name]
```

```
set cells [lsort -u $cells] 对单元格列表进行排序，并去除重复项
```

```
foreach cell $cells {
```

```
    puts $hcell_file "$cell $cell"
```

```
    puts $hcell_list_ir "$cell"
```

```
}
```

```
close $hcell_file
```

```
close $hcell_list_ir
```

hcell_file用于LVS验证，
hcell_list_ir用于IR-drop分析。

循环遍历单元格列表，并将每个单元格的名称
写入两个文件中。在hcell_file中，每行由单元
格名称和单元格名称组成，以便LVS验证使用。
在hcell_list_ir中，每行只包含单元格名称，以
便IR-drop分析使用

./scripts/pr/chip_done.tcl

布局和布线（九）



- Chip Finish：运行脚本讲解8

这段代码的作用是将物理布局导出为GDS文件

```
# -----  
# Gds streamOut 设置了streamOut命令的输出模式  
# -----  
setStreamOutMode -textSize 5 设置文本大小为5  
setStreamOutMode -virtualConnection true 启用虚拟连接  
setStreamOutMode -uniquifyCellNamesPrefix true 添加单元格名称前缀以确保唯一性  
setStreamOutMode -check_map_file true 检查GDS映射文件以确保正确性  
streamOut $::env(RESULT_DIR)/pr/data/ibex_core.gds -mapFile $::env(GDS_MAP_FILE)  
-libName DesignLib -units 1000 -mode ALL streamOut命令将物理布局导出为GDS文件  
saveDesign $::env(RESULT_DIR)/pr/data/chip_done.enc  
exit
```

./scripts/pr/chip_done.tcl

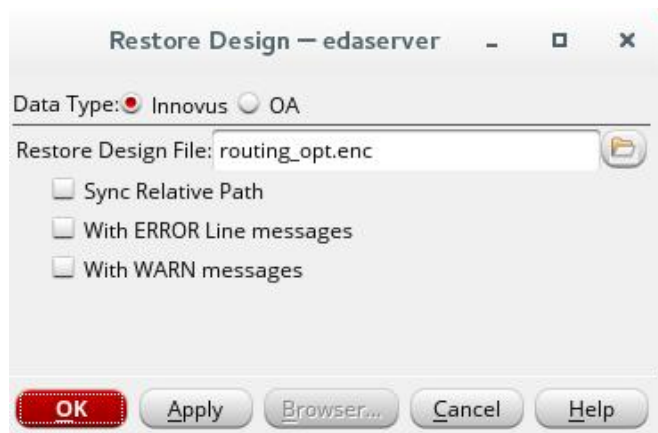
\$::env(RESULT_DIR)/pr/data/ibex_core.gds是导出的GDS文件的路径

\$::env(GDS_MAP_FILE)是GDS映射文件的路径，DesignLib是库名称

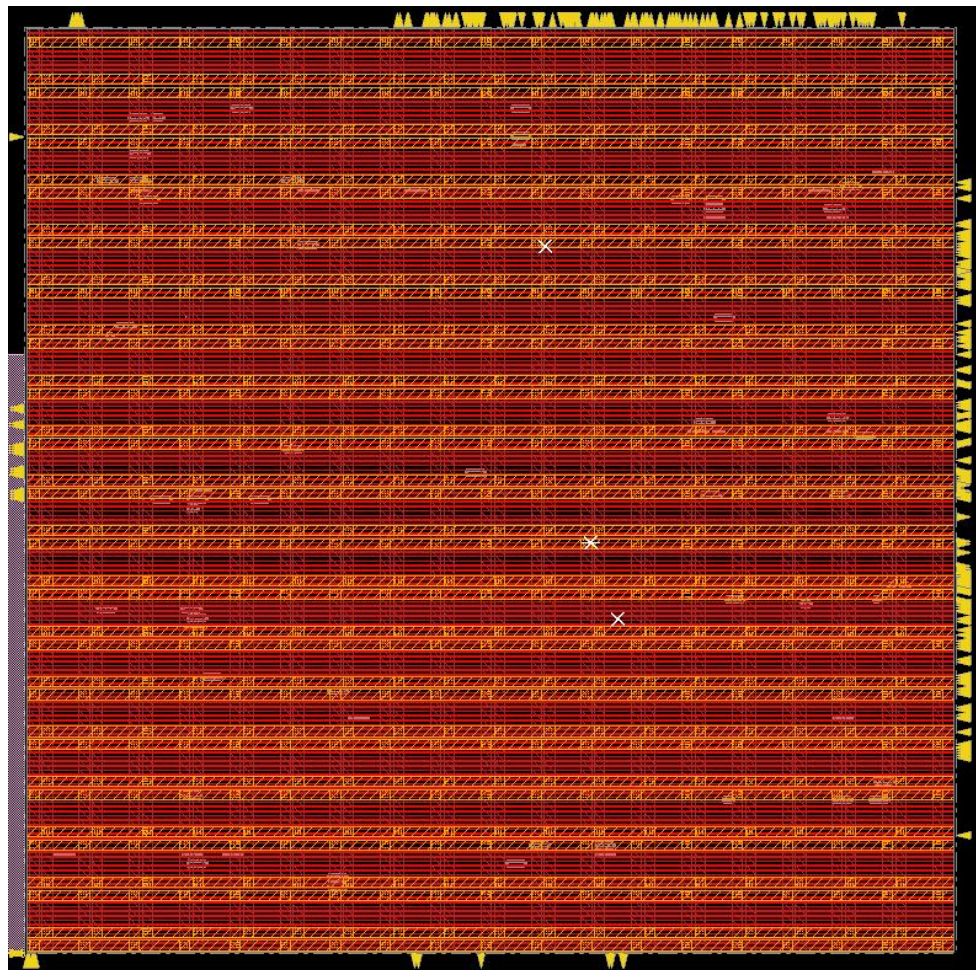
布局 and 布线 (九)



- Chip Finish:



可以打开chip_done.enc查看芯片情况!



布局和布线（九）



- 小结

- CTS : 时钟树综合
- Post-CTS : 时钟树综合后进行优化
- Routing : 进行布线，满足时序要求
- Post-Routing: 布线优化，更精致的布线，减少glitch冲突
- Chip_finish: 输出gds文件

